Instituto Tecnológico de Costa Rica
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC-8071 Seguridad del Software GR 2
Prof. Erika Marin Schumann
I Semestre 2026

Proyecto 1 - Simulación de Atención de Procesos de un Sistema Operativo

Felipe Javier Murillo Chaves - 2023083005
Kevin Núñez Camacho - 2023198499
Axel López Vega - 2023094841

Cartago, 13 de abril del 2026

Tabla de contenidos

Tabla de contenidos **2**
Estrategia de solución **3**
Análisis de Resultados **5**
Lecciones aprendidas **6**
Casos de prueba **7**
1. Conexión básica cliente-servidor 7
2. Validación de prioridad (cliente) 7
3. FIFO (orden por llegada/PID) 7
4. SJF (Shortest Job First) 8
5. HPF (Prioridad) 8
6. Round Robin 8
7. CPU ocioso 9
8. Múltiples clientes 9
9. Cierre del servidor 9
Comparación: Investigación comparativa entre los hilos de Java y PThreads **10**
Manual de usuario **11**
Requisitos 11
Para correr el proyecto 11
Bitácora de trabajo **13**
Bibliografía **14**

Estrategia de solución

El proyecto se desarrolló mediante un sistema cliente-servidor donde la comunicación se hacía por medio de sockets. La solución se divide en dos programas independientes, Cliente.C y Servidor.C

Cliente.C
El cliente es responsable de generar procesos y enviarlos al servidor. Cada proceso se maneja como un hilo independiente que abre su propia conexión al servidor, este envía los datos del proceso (burst y prioridad) y espera el ACK() con el PID(Process Identifier) asignado. Al iniciar el cliente solicita al usuario el modo de operación (si manual o automático) y el algoritmo de planificación (FIFO, SJF, HPF o Round Robin).

En modo manual el cliente lee el archivo Procesos.txt línea por línea, donde cada línea contiene el burst y la prioridad de un proceso. Entre cada lectura se aplica un sleep aleatorio de 3 a 8 segundos. En modo automático el cliente genera procesos de forma continua con valores aleatorios de burst (dentro de un rango definido por el usuario) y prioridad (1 a 10), con un intervalo aleatorio entre creaciones hasta que el usuario presiona Ctrl+C.

Servidor.C
El servidor implementa el simulador del sistema operativo, contiene dos hilos principales:

- Job Scheduler Thread: Este hilo escucha continuamente el socket del servidor aceptando conexiones entrantes. Cuando recibe un proceso, se le asigna un PID incremental y construye su PCB (Process Control Block) con los campos: pid, burstTotal, burstRestante, prioridad, llegada, salida, TAT y WT. Se inserta en la cola Ready y envía el ACK al cliente con el PID asignado.
- CPU Scheduler Thread: Este hilo verifica constantemente si hay procesos en la cola Ready. Utiliza una variable de condición para evitar busy-waiting. Si la cola está vacía, el hilo se bloquea esperando una señal. Si transcurre un segundo sin recibir señal, se acumula un segundo de CPU ocioso. Cuando hay un proceso disponible, se selecciona el siguiente según el algoritmo configurado y lo ejecuta simulando el burst con un sleep.

Cada proceso se representa con la siguiente estructura:
pid | burstTotal | burstRestante | prioridad | llegada | salida | TAT | WT

Se implementaron los siguientes procesos:

- FIFO (First In First Out): Selecciona el proceso con menor PID disponible en la cola Ready.
- SJF (Shortest Job First): Recorre la lista de procesos en espera y selecciona el que tiene el menor burstRestante
- HPF (Highest Priority First): Selecciona el proceso con el valor más bajo en el campo prioridad. En caso de empate, se aplica FIFO.

- Round Robin: Extrae el primer proceso de la cola en orden FIFO y lo ejecuta durante el quantum configurado. Si el proceso no termina, se reinserta al final de la cola con el burstRestante actualizado.

Al salir, el servidor detiene los hilos, espera a que terminen y genera un reporte final que incluye: número de procesos ejecutados, segundos de CPU ocioso, tabla de TAT y WT para cada proceso y el promedio de waiting time.

Análisis de Resultados

Actividad	% Completado	Observaciones
Comunicación cliente-servidor mediante sockets	100%	Implementado correctamente. Cada proceso abre su propia conexión.
Envío de algoritmo y quantum al servidor al inicio	100%	El cliente envía primero una conexión dedicada con el algoritmo y quantum.
Modo Manual (lectura desde archivo Procesos.txt)	100%	Se valida formato, rango de prioridad y se aplica sleep aleatorio entre procesos.
Modo Automático (generación aleatoria de procesos)	100%	Generación continua hasta Ctrl+C con parámetros de burst configurables.
Un hilo por proceso en el cliente	100%	Cada proceso es un pthread que conecta, envía y espera el ACK.
Job Scheduler Thread (asignación de PID y PCB)	100%	Asigna PID atómico, construye PCB, inserta en cola y envía ACK.
CPU Scheduler Thread con variable de condición	100%	Uso correcto de pthread_cond_timedwait para evitar busy-waiting.
Proceso FIFO	100%	Selección del menor PID en la cola Ready.
Proceso SJF	100%	Selección del menor burstRestante
Proceso HPF	100%	Selección por menor valor de prioridad, FIFO en empate.

Proceso Round Robin	100%	Manejo correcto del burstRestante y reinserción al final de la cola.
Conteo de llegada y salida de procesos	–	Existe una inconsistencia porque el tiempo de llegada y salida de cada proceso depende de la función time(null) que dedica un tiempo en segundos sin tomar décimas a una variable.
Comando para visualizar estado de la cola Ready	100%	Disponible en cualquier momento durante la ejecución del servidor
Comando para detener el servidor	100%	Cierra sockets, espera hilos y genera reporte.
Conteo de CPU ocioso	100%	Acumulado mediante timedwait con timeout de 1 segundo.
Reporte final	100%	Impreso al cerrar el servidor con todos los procesos finalizados

Lecciones aprendidas

- Manejo de señales en entornos multihilo: Descubrimos que las señales como SIGINT pueden interrumpir llamadas como accept() o sleep(), devolviendo EINTR. Es necesario verificar este código de error y reintentar la operación cuando el servidor sigue activo.
- Sockets y arquitectura cliente-servidor: Comprendimos en profundidad el flujo de socket(), bind(), listen(), accept() y connect(). Un aprendizaje importante fue que cada proceso cliente debe abrir su propia conexión.
- Liberación de recursos: Aprendimos la importancia de liberar toda la memoria dinámica (PCB) y cerrar todos los file descriptors de sockets tanto en el cliente como en el servidor para evitar fugas de memoria y descriptores huérfanos.
- Diseño del PCB: Definir bien los campos del PCB desde el inicio (incluyendo llegada, salida, TAT y WT) simplificó el cálculo del reporte final. Tener que agregar campos después habría requerido rehacer múltiples funciones.
- Separación del Job Scheduler y CPU Scheduler: Mantener ambos schedulers como hilos independientes simplificó la especificación y facilitó la comprensión del flujo del sistema operativo simulado.

Casos de prueba

1. Conexión básica cliente-servidor

Objetivo: verificar que el cliente se conecta y el servidor responde

Pasos:

1. Ejecutar servidor
2. Ejecutar cliente
3. Enviar un proceso

Resultado esperado:

- El cliente muestra: *PID asignado*
- El servidor recibe el proceso

```
axel@debian:~/Documentos/SO/Sistemas-Operativos/Proyecto 1$ ./Servidor.out 3000
```

```
axel@debian:~/Documentos/SO/Sistemas-Operativos/Proyecto 1$ ./Cliente.out 127.0.0.1 3000
Seleccione modo cliente:
1. Manual
2. Automatico
```

2. Validación de prioridad (cliente)

En tu código:

- Prioridad válida: 1 a 10

Caso:

- Enviar prioridad = 15

Resultado esperado:

- El cliente dice:
  Proceso ignorado

```
Rango de burst permitido (Automatico)
Burst minimo: 1
Burst maximo: 15
Modo automatico activo. Presione Ctrl+C para detener.
Proceso enviado (Burst=6, Prioridad=2) -> PID asignado: 1
Proceso enviado (Burst=4, Prioridad=1) -> PID asignado: 2
Proceso enviado (Burst=9, Prioridad=2) -> PID asignado: 3
Proceso enviado (Burst=11, Prioridad=5) -> PID asignado: 4
Proceso enviado (Burst=8, Prioridad=4) -> PID asignado: 5
Proceso enviado (Burst=9, Prioridad=6) -> PID asignado: 6
```

En forma automática, la prioridad se mantiene en un rango establecido. No falla.

En forma manual, se establecen en el .txt

3. FIFO (orden por llegada/PID)

Objetivo: verificar orden de ejecución

Entrada:

Proceso 1 → burst 5
Proceso 2 → burst 3
Proceso 3 → burst 2

Resultado esperado:

- Se ejecutan en orden de llegada

```
Seleccione algoritmo de planificacion:
1. FIFO
2. SJF
3. HPF
4. Round Robin
1
Proceso enviado (Burst=10, Prioridad=4) -> PID asignado: 1
Proceso enviado (Burst=1, Prioridad=10) -> PID asignado: 2
Proceso enviado (Burst=5, Prioridad=3) -> PID asignado: 3
Proceso enviado (Burst=2, Prioridad=2) -> PID asignado: 4
```

```
Algoritmo de planificacion recibido: 1
Servidor activo en puerto 3000. Comandos: cola | salir
Proceso PID=1 entra en ejecucion (Llegada=0, Burst=10, Prioridad=4)
PID=1 tramo inicia en t=0 y ejecuta 10 segundos
PID=1 tramo termina en t=10
Proceso PID=1 finalizado. Salida=10 TAT=10 WT=0
Proceso PID=2 entra en ejecucion (Llegada=3, Burst=1, Prioridad=10)
PID=2 tramo inicia en t=10 y ejecuta 1 segundos
PID=2 tramo termina en t=11
Proceso PID=2 finalizado. Salida=11 TAT=8 WT=7
Proceso PID=3 entra en ejecucion (Llegada=7, Burst=5, Prioridad=3)
PID=3 tramo inicia en t=11 y ejecuta 5 segundos
PID=3 tramo termina en t=16
Proceso PID=3 finalizado. Salida=16 TAT=9 WT=4
Proceso PID=4 entra en ejecucion (Llegada=11, Burst=2, Prioridad=2)
PID=4 tramo inicia en t=16 y ejecuta 2 segundos
PID=4 tramo termina en t=18
Proceso PID=4 finalizado. Salida=18 TAT=7 WT=5
```

4. SJF (Shortest Job First)

Objetivo: verificar selección por menor burst

Entrada:

P1 → burst 10
P2 → burst 2
P3 → burst 5

Resultado esperado:
Orden:

P2 → P3 → P1

```
Seleccione algoritmo de planificacion:
1. FIFO
2. SJF
3. HPF
4. Round Robin
2
Proceso enviado (Burst=10, Prioridad=4) -> PID asignado: 6
Proceso enviado (Burst=1, Prioridad=10) -> PID asignado: 7
Proceso enviado (Burst=5, Prioridad=3) -> PID asignado: 8
Proceso enviado (Burst=2, Prioridad=2) -> PID asignado: 9
```

```
Proceso PID=6 entra en ejecucion (Llegada=97, Burst=10, Prioridad=4)
PID=6 tramo inicia en t=99 y ejecuta 10 segundos
PID=6 tramo termina en t=109
Proceso PID=6 finalizado. Salida=109 TAT=12 WT=2
Proceso PID=7 entra en ejecucion (Llegada=101, Burst=1, Prioridad=10)
PID=7 tramo inicia en t=109 y ejecuta 1 segundos
PID=7 tramo termina en t=110
Proceso PID=7 finalizado. Salida=110 TAT=9 WT=8
Proceso PID=8 entra en ejecucion (Llegada=104, Burst=5, Prioridad=3)
PID=8 tramo inicia en t=110 y ejecuta 5 segundos
PID=8 tramo termina en t=115
Proceso PID=8 finalizado. Salida=115 TAT=11 WT=6
Proceso PID=9 entra en ejecucion (Llegada=111, Burst=2, Prioridad=2)
PID=9 tramo inicia en t=115 y ejecuta 2 segundos
PID=9 tramo termina en t=117
Proceso PID=9 finalizado. Salida=117 TAT=6 WT=4
```

5. HPF (Prioridad)

Objetivo: verificar prioridad más alta (número menor)

Entrada:

P1 → prioridad 5
P2 → prioridad 1
P3 → prioridad 3

Resultado esperado:

P2 → P3 → P1

```
Proceso PID=11 entra en ejecucion (Llegada=320, Burst=10, Prioridad=4)
PID=11 tramo inicia en t=323 y ejecuta 10 segundos
PID=11 tramo termina en t=333
Proceso PID=11 finalizado. Salida=333 TAT=13 WT=3
Proceso PID=12 entra en ejecucion (Llegada=325, Burst=1, Prioridad=10)
PID=12 tramo inicia en t=333 y ejecuta 1 segundos
PID=12 tramo termina en t=334
Proceso PID=12 finalizado. Salida=334 TAT=9 WT=8
Proceso PID=13 entra en ejecucion (Llegada=332, Burst=5, Prioridad=3)
PID=13 tramo inicia en t=334 y ejecuta 5 segundos
PID=13 tramo termina en t=339
Proceso PID=13 finalizado. Salida=339 TAT=7 WT=2
Proceso PID=14 entra en ejecucion (Llegada=339, Burst=2, Prioridad=2)
PID=14 tramo inicia en t=339 y ejecuta 2 segundos
PID=14 tramo termina en t=341
Proceso PID=14 finalizado. Salida=341 TAT=2 WT=0
```

```
Seleccione algoritmo de planificacion:
1. FIFO
2. SJF
3. HPF
4. Round Robin
3
Proceso enviado (Burst=10, Prioridad=4) -> PID asignado: 11
Proceso enviado (Burst=1, Prioridad=10) -> PID asignado: 12
Proceso enviado (Burst=5, Prioridad=3) -> PID asignado: 13
Proceso enviado (Burst=2, Prioridad=2) -> PID asignado: 14
```

6. Round Robin

Objetivo: verificar uso de quantum

Entrada:

- Quantum = 2
- P1 → burst 5

Resultado esperado:

Ejecuta 2 → queda 3
Ejecuta 2 → queda 1
Ejecuta 1 → termina

El código lo realiza en esta sección

- reduce burst y reingresa a READY

```
Proceso PID=16 entra en ejecucion (Llegada=439, Burst=10, Prioridad=4)
PID=16 tramo inicia en t=443 y ejecuta 10 segundos
PID=16 tramo termina en t=453
Proceso PID=16 finalizado. Salida=453 TAT=14 WT=4
Proceso PID=17 entra en ejecucion (Llegada=445, Burst=1, Prioridad=10)
PID=17 tramo inicia en t=453 y ejecuta 1 segundos
PID=17 tramo termina en t=454
Proceso PID=17 finalizado. Salida=454 TAT=9 WT=8
Proceso PID=18 entra en ejecucion (Llegada=450, Burst=5, Prioridad=3)
PID=18 tramo inicia en t=454 y ejecuta 5 segundos
PID=18 tramo termina en t=459
Proceso PID=18 finalizado. Salida=459 TAT=9 WT=4
Proceso PID=19 entra en ejecucion (Llegada=455, Burst=2, Prioridad=2)
PID=19 tramo inicia en t=459 y ejecuta 2 segundos
PID=19 tramo termina en t=461
Proceso PID=19 finalizado. Salida=461 TAT=6 WT=4
```

```
Seleccione algoritmo de planificacion:
1. FIFO
2. SJF
3. HPF
4. Round Robin
4
Ingrese el quantum (debe ser mayor a 0): 5
Proceso enviado (Burst=10, Prioridad=4) -> PID asignado: 16
Proceso enviado (Burst=1, Prioridad=10) -> PID asignado: 17
Proceso enviado (Burst=5, Prioridad=3) -> PID asignado: 18
Proceso enviado (Burst=2, Prioridad=2) -> PID asignado: 19
```

7. CPU ocioso

Objetivo: verificar que cuenta tiempo sin procesos

Pasos:

1. Ejecutar servidor
2. No enviar procesos por unos segundos

Resultado esperado:

- CPU ocioso aumenta

```
===== REPORTE FINAL SERVIDOR =====
Procesos ejecutados: 19
CPU ocioso (segundos): 570
```

En este caso, es porque se mantuvo abierto para las pruebas

8. Cierre del servidor

Pasos:

- Escribir `salir` en servidor

Resultado esperado:

- Muestra reporte final
- Libera memoria

```
Comando no reconocido. Use: cola | salir
salir

===== REPORTE FINAL SERVIDOR =====
Procesos ejecutados: 19
CPU ocioso (segundos): 570

Tabla TAT/WT:
PID     Llegada Burst   Salida  TAT     WT
19      455     2       461     6       4
18      450     5       459     9       4
17      445     1       454     9       8
16      439     10      453     14      4
15      439     4       443     4       0
14      339     2       341     2       0
13      332     5       339     7       2
12      325     1       334     9       8
11      320     10      333     13      3
10      320     3       323     3       0
9       111     2       117     6       4
```

Comparación: Investigación comparativa entre los hilos de Java y PThreads

En términos de **USABILIDAD**

Aspecto	Hilos de Java	PThreads
Abstracción	Alto: el lenguaje oculta la complejidad del sistema operativo. Crear y lanzar un hilo es sencillo e intuitivo.	Bajo: el programador trabaja directamente con las funciones del sistema. Requiere mayor conocimiento del entorno.
Facilidad de uso	Sencillo para el programador: manejo automático de pilas y tipos.	Más complejo, se manejan punteros, memoria dinámica y castings manuales.
Sincronización	Ofrece mecanismos integrados en el lenguaje y bibliotecas de alto nivel con semáforos, barreras y colas concurrentes.	Se utilizan primitivas de sincronización del sistema operativo, exclusión mutua y variables de condición.
Manejo de errores	Se usa el mecanismo de excepciones del lenguaje, lo cual es más expresivo y fácil de leer.	Se verifican códigos de retorno enteros en cada llamada. Requiere manejo explícito y constante.
Portabilidad	Alta: el programa corre en cualquier sistema con JVM sin cambios.	Media: sigue el estándar POSIX, pero depende del sistema operativo donde se compila y ejecuta.

Detalles tecnicos

Los hilos de Java son gestionados por la máquina virtual, esta se encarga de mapearlos a los hilos del sistema operativo de forma transparente, esto significa que el programador no tiene que preocuparse por la interacción directa con el kernel. En cambio, PThreads trabaja directamente con el sistema operativo, osea que cada hilo creado es un hilo real del kernel, lo que da mayor visibilidad y control sobre lo que está ocurriendo a nivel del sistema. La forma en la que se pasa información a los hilos también cambia mucho. Java permite pasar objetos fuertemente tipados a través de los constructores del hilo o por medio de variables capturadas. En C con PThreads, los datos se pasan como un puntero que debe ser interpretado y casteado manualmente dentro de la función del hilo, y esto introduce un mayor riesgo de errores si no se maneja con cuidado. Otro punto importante es la gestión de memoria, en Java, el recolector de basura libera automáticamente los objetos que ya no se usan, incluyendo los recursos asociados a los hilos. En C, el programador debe liberar explícitamente toda la memoria que reservó de forma dinámica, si no se hace correctamente, se producen salidas de memoria que pueden ser difíciles de detectar. En este proyecto, por ejemplo, fue necesario liberar manualmente cada PCB al finalizar su ejecución y liberar las listas enlazadas al cerrar el

servidor. Finalmente, la cancelación o detención de hilos también presenta diferencias. Java proporciona mecanismos integrados para interrumpir un hilo de forma segura. En PThreads, la manera más limpia es utilizar variables de control compartidas que el hilo verifica periódicamente, como se hizo en este proyecto con la variable servidorActivo para detener el servidor y generarAutomatico para el cliente en modo automático.

Rendimiento
En términos de rendimiento, los PThreads tienden a ser mucho más eficientes en escenarios de bajo nivel porque trabajan directamente sobre los recursos del sistema operativo. Java al correr en una máquina virtual introduce overhead que se puede notar en aplicaciones con muchos hilos o un uso intensivo de memoria. Pero en la práctica esta diferencia raramente significa algo, esto porque Java compensa con herramientas de alto nivel para el manejo de hilos que hacen el desarrollo más seguro y menos propenso a errores.

Para este proyecto el uso de PThreads fue bastante apropiado ya que el lenguaje requerido es C y el entorno linux. PThreads ofrece control directo sobre los recursos del sistema y esto es necesario para simular un planificador del CPU. Java habría simplificado la implementación pero a costo de perder visibilidad sobre los mecanismos de bajo nivel que son el objetivo de este proyecto.

Manual de usuario

Requisitos

- Sistema operativo: Linux (Ubuntu 20.04 o superior recomendado)
- Compilador: GCC (GNU Compiler Collection) versión 9 o superior
- Librería PThreads (incluida por defecto en Linux)

Para correr el proyecto

Se abre una terminal en el directorio del proyecto y se ejecutan los siguientes comandos:

- Compilar el servidor: gcc -o Servidor Servidor.c -lpthread
- Compilar el cliente: gcc -o Cliente Cliente.c -lpthread
- El servidor se inicia primero y recibe el número de puerto: ./Servidor <puerto>
    - Ejemplo: ./Servidor 8080

Una vez iniciado, el servidor queda en espera de la primera conexión del cliente (que enviará el algoritmo y quantum). Después de recibirlos, el servidor muestra el algoritmo seleccionado y queda activo. Estos son los comandos disponibles durante la ejecución del servidor:

- cola: Muestra el estado actual de la cola Ready con todos los procesos en espera.
- salir: Detiene el servidor y muestra el reporte final hasta el momento de ejecución del comando salir.

Luego se ejecuta el cliente:

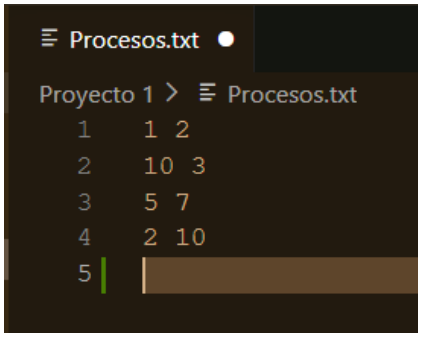
- El cliente recibe el host del servidor y el puerto: ./Cliente <host> <puerto>
    - ./Cliente localhost 8080

El cliente solicitará al usuario seleccionar:

- El modo de operación: 1 = Manual, 2 = Automático
- El algoritmo de planificación: 1 = FIFO, 2 = SJF, 3 = HPF, 4 = Round Robin
- Si seleccionó Round Robin, el valor del quantum (entero positivo)
- Si seleccionó modo Automático, el rango mínimo y máximo del burst.

Para modo manual:

El archivo debe contener una línea por proceso con dos valores separados por espacio



La prioridad debe estar entre 1 (mínima) y 10 (máxima). Las líneas con valores fuera de rango son ignoradas con un mensaje de advertencia. En este caso, el primer número de cada línea pertenece al burst de cada proceso y separado por un espacio el segundo número pertenece a la prioridad.

Para modo automatico:

Se debe primero seleccionar el algoritmo que desee ejecutar, y luego el sistema le pedirá un rango de burst para comenzar a ejecutar profesos y en el momento que el usuario crea pertinente para la creación de procesos debe presionar ctrl+c. Para luego ya ver la ejecución en el servidor

Servidor:

Una vez estando en el servidor el usuario podrá ver la simulación de la ejecución de los procesos y en cualquier momento colocar los comandos cola o salir, para hacer las siguientes acciones:

- **Detener el servidor:**
    - Para detener el servidor, se escribe el comando 'salir' en la terminal, o se presiona Ctrl+C. En ambos casos el servidor mostrará el reporte final antes de terminar. También se realiza Ctrl+C para detener el cliente en la terminal de el.

- **Consultar el servidor:**

- Para consultar la cola en el momento que se requiera por el usuario solo en la terminal del servidor debe escribir “cola” y darle enter para que en este caso se impriman los procesos que se encuentran en ready

Bitácora de trabajo

Primera semana de trabajo

Fecha	Actividades realizadas
23/03/2026	Lectura y análisis detallado de la especificación del proyecto. Identificación de los componentes principales: cliente, servidor, Job Scheduler, CPU Scheduler.
24/03/2026	Investigación sobre sockets en C (socket, bind, listen, accept, connect, send, recv). Investigación sobre PThreads (pthread_create, pthread_join, mutex, cond).
25/03/2026	Diseño de las estructuras de datos: MensajeProceso, MensajeAck, PCB. Definición del protocolo de comunicación entre cliente y servidor.
26/03/2026	- - -
27/03/2026	- - -

Segunda semana de trabajo

Fecha	Actividades realizadas
30/03/2026	Implementación del CPU Scheduler Thread con pthread_cond_timedwait para manejo de CPU ocioso. Prueba con algoritmo FIFO básico.
31/03/2026	Implementación de los algoritmos SJF y HPF.
01/04/2026	Implementación del algoritmo Round Robin con manejo de burstRestante y reinserción a la cola. Pruebas con quantum variable.
02/04/2026	- - -
03/04/2026	- - -

Tercera semana de trabajo

Fecha	Actividades realizadas
06/04/2026	Implementación del modo manual del cliente y modo automático
07/04/2026	Implementación del reporte final: procesos ejecutados, CPU ocioso, tabla TAT/WT y promedio WT. Validación de valores con cálculos manuales.
08/04/2026	Pruebas de liberación de memoria y cierre de sockets. Verificación con valgrind de ausencia de memory leaks. Refinamiento de mensajes en consola.
09/04/2026	Redacción de la documentación
10/04/2026	Redacción de la documentación

Bibliografía

C. Roxas, “Procesos e hilos en C de Unix/Linux.” Available: https://old.chuidiang.org/clinux/procesos/procesoshilos.php

G. Ippolito, “Linux tutorial: POSIX threads.” Available: https://www.cs.cmu.edu/afs/cs/academic/class/15492-f07/www/pthreads.html

ProgramaTutos, “Como crear hilos en C usando Pthread (POSIX),” *YouTube*. Sep. 20, 2023. Available: https://www.youtube.com/watch?v=ROzCbSwrltA

Jeisson Hidalgo-Céspedes , Allan Berrocal-Rojas , “Programación concurrente: ejemplos de clase.” Available: https://jeisson.ecci.ucr.ac.cr/concurrente/2020c/ejemplos/

dieguiariel, “Progamas Ejemplo Sockets en C en Linux Cliente Servidor,” *YouTube*. Sep. 05, 2020. Available: https://www.youtube.com/watch?v=AfDfmsh8OGo

Iván Morelos Dorantes, “Aprendiendo a programar en C #55 - Leer archivos por líneas,” *YouTube*. Feb. 18, 2016. Available: https://www.youtube.com/watch?v=8d8Py6ucxPI

Iván Morelos Dorantes, “Aprendiendo a programar en C #55 - Leer archivos por líneas,” *YouTube*. Feb. 18, 2016. Available: https://www.youtube.com/watch?v=8d8Py6ucxPI

FastRB development, “Uso de Switch case en lenguaje C,” *YouTube*. Apr. 18, 2023. Available: https://www.youtube.com/watch?v=jD9LaCNqk1g

N. N, “Listas en C,” *Monografias.com*, Dec. 29, 2011. Available: https://www.monografias.com/trabajos90/listas-c/listas-c

P. G. Ruiz, “Colas y colas circulares en C.” Available: http://www.pedrogonzalezruiz.net/colas/colas.html

StackOverflow, “Java Threads vs Pthreads”, Mar. 13, 2011. Available: https://stackoverflow.com/questions/5269535/java-threads-vs-pthreads

Quora, “Can you explain the differences between POSIX threads and Java threads? In what situations would one be more suitable than the other?”. Available: https://www.quora.com/Can-you-explain-the-differences-between-POSIX-threads-and-Java-threads-In-what-situations-would-one-be-more-suitable-than-the-other

GeeksForGeeks, “Difference Between Java Threads and OS Threads”, Jul. 23, 2025. Available: https://www.geeksforgeeks.org/java/difference-between-java-threads-and-os-threads/